

UNIVERSIDADE FEDERAL DE LAVRAS
DISCIPLINA DE ENGENHARIA DE SOFTWARE

PROSPECÇÃO TEÓRICA E TECNOLÓGICA

Aplicação de Design Tokens na Padronização de Interfaces Digitais

Tainara de Fátima Matias Souza

Link do Repositório no GitHub: [tainarafms/PTT-Design-Tokens](https://github.com/tainarafms/PTT-Design-Tokens)

Link do GitHub Pages: <https://tainarafms.github.io/PTT-Design-Tokens/>

Introdução e Contextualização

Um Design System atua como uma biblioteca centralizada de padrões visuais, componentes reutilizáveis e diretrizes de estilo adotadas por uma empresa para garantir consistência em seus produtos. Dentro desse ecossistema altamente estruturado operam os Design Tokens, que consistem nas menores unidades de estilo configuradas como variáveis técnicas. Essas variáveis armazenam decisões de interface, como cores, tipografia e espaçamentos, substituindo valores estáticos e repetitivos no código por nomes semânticos que podem ser lidos de forma padronizada por qualquer linguagem de programação.

Essa tecnologia é extremamente relevante para o cenário atual da Engenharia de Software devido à complexidade crescente no desenvolvimento de interfaces digitais. Com aplicações precisando rodar simultaneamente em diversos dispositivos e navegadores web, manter o visual previsível e tecnicamente escalável tornou-se uma exigência indispensável. O uso de tokens cria uma fundação sólida e uma linguagem comum entre o design e a programação do front-end, garantindo que qualquer decisão visual abstrata seja traduzida de forma coesa em todo o ciclo de vida do projeto.

A abordagem tradicional de estilização impõe uma enorme dor para a equipe técnica devido à repetição massiva de valores espalhados por dezenas de arquivos. Essa prática gera inconsistência visual, acúmulo de débitos técnicos e dificulta drasticamente a manutenção adaptativa do software, como na exigência de implementação de um tema claro e escuro (Light/Dark Mode). Quando uma marca precisa alterar sua paleta de cores, buscar e substituir códigos hexadecimais manualmente consome tempo e abre margem para falhas humanas críticas.

Os Design Tokens resolvem esse exato problema ao centralizar todos os estilos na raiz estrutural do projeto. Através dessa técnica, a interface passa a ser tratada com o mesmo rigor aplicado ao back-end, estabelecendo uma única fonte de verdade para a aplicação. Dessa forma, uma simples alteração no valor da variável-raiz propagará a tematização dinâmica por todo o sistema instantaneamente, provando a facilidade de manutenção, reduzindo o tempo de retrabalho do desenvolvedor e diminuindo o custo para a empresa.

Fundamentação Teórica

Historicamente, o desenvolvimento de interfaces digitais passou por uma fase de grande inconsistência visual, na qual o design era transferido para o código de forma manual e desestruturada. Para resolver isso, a indústria evoluiu para a criação de grandes Design Systems abertos, consolidando referências principais como o Material Design da Google, o Carbon da IBM e o Fluent da Microsoft. O conceito de Design Tokens surgiu nesse exato cenário, criado inicialmente pela equipe da Salesforce. O objetivo era garantir que as decisões visuais fossem escaláveis e agnósticas, permitindo traduzir o design para qualquer linguagem de código de forma 100% automatizada.

Tecnicamente, os tokens funcionam como um dicionário de dados centralizado que atua como a única fonte da verdade do projeto, operando de forma estruturada em três camadas principais de abstração. A primeira abrange os tokens globais ou primitivos, que armazenam o valor-base absoluto de uma cor. A segunda contém os tokens semânticos, que dão um contexto prático para aquele valor numérico. Por fim, a terceira camada é formada pelos tokens de componente, que são aplicados diretamente nos botões e demais elementos interativos da tela pelo desenvolvedor *front-end*.

Quando ocorre uma mudança na identidade visual, a equipe altera apenas a variável-raiz e o sistema central garante a integridade da atualização em todas as telas, viabilizando a tematização dinâmica. Para ilustrar de forma esquemática essa arquitetura operando na prática, o fluxo de uma decisão visual segue uma ordem lógica. A decisão de design alimenta o token global, que por sua vez fornece o valor para o token semântico, sendo este finalmente aplicado no token de componente visível para o usuário final. O diagrama abaixo detalha esse fluxo de propagação:

Diagrama Esquemático: Fluxo de Propagação dos Design Tokens



Conceitos-Chave:

- **Design System:** Biblioteca centralizada de padrões visuais, componentes reutilizáveis e diretrizes de estilo de uma empresa.
- **Design Tokens:** Menores unidades de estilo configuradas como variáveis técnicas que armazenam decisões de interface, substituindo valores estáticos e soltos no código.
- **Fonte Única da Verdade:** Princípio arquitetural onde toda decisão de design existe em apenas um lugar centralizado, evitando a duplicação de dados.
- **Tematização Dinâmica:** Capacidade de alternar instantaneamente entre temas visuais, como o modo claro e o modo escuro, alterando exclusivamente as variáveis na raiz da aplicação.
- **Manutenção de Software:** Etapa do ciclo de vida em que o sistema sofre modificações para corrigir erros, adaptar-se a mudanças ou incorporar melhorias. A centralização do estilo facilita e agiliza drasticamente as adaptações visuais nesta fase.
- **Débito Técnico:** Consequência do acúmulo de implementações desorganizadas e retrabalhos que dificultam a evolução do software. A aplicação de padrões de design visa reduzir a dívida técnica do sistema.

Análise Comparativa

A comparação entre métodos revela que a abordagem tradicional de estilização depende da inserção manual de valores estáticos diretamente nos arquivos. Em um cenário comum usando CSS clássico, cores e espaçamentos ficam engessados no código, tornando qualquer manutenção adaptativa extremamente lenta e propensa a falhas críticas. Outra alternativa histórica muito utilizada são os pré-processadores de estilo, como o SASS(Syntactically Awesome Style Sheets), que introduziram o uso prático de variáveis. Contudo, essas variáveis tradicionais operam restritas a um único ecossistema, não resolvendo o problema de integração quando o software exige versões simultâneas para a *web* e para os dispositivos móveis.

Como estado da arte, os Design Tokens superam essas alternativas operando de fato como uma camada de dados totalmente agnóstica de plataforma. Enquanto o método tradicional prende o desenvolvedor a uma linguagem de marcação específica, os tokens são estruturados em um formato universal independente, geralmente em arquivos JSON.

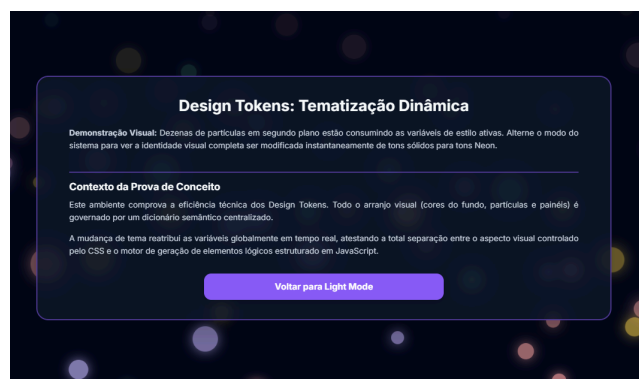
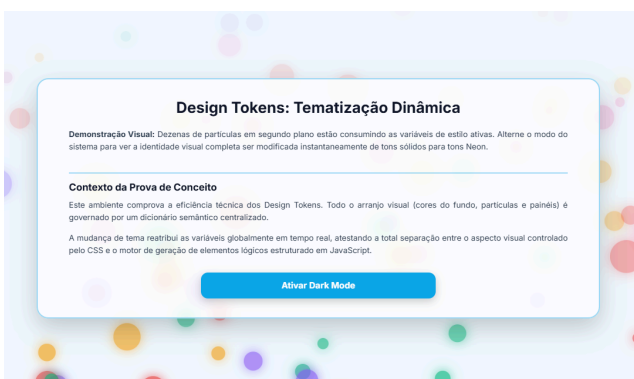
A partir dessa raiz padronizada, ferramentas de automação leem os dados brutos e os convertem em variáveis CSS para a *web*, arquivos XML para o Android e classes Swift para o iOS de forma simultânea. Essa fundação centralizada garante uma sincronia técnica em múltiplos ambientes que seria humanamente impossível no método tradicional.

A principal vantagem técnica dessa arquitetura é a drástica redução do acúmulo de débitos técnicos e o aumento da coesão estrutural do software. O ganho de produtividade para a equipe técnica é inegável, pois o desenvolvedor *front-end* passa a focar estritamente na lógica dos componentes, eliminando o tempo perdido ajustando estilos repetitivos manualmente. Essa centralização permite que alterações visuais de altíssima complexidade sejam executadas com total segurança técnica e previsibilidade, garantindo rastreabilidade dos estilos e facilitando atualizações de mercado em larga escala com um impacto quase nulo no cronograma de desenvolvimento do projeto.

Contudo, demonstrando capacidade crítica analítica e fugindo de discursos comerciais otimistas, é vital reconhecer as limitações práticas dessa tecnologia. A implementação de uma arquitetura baseada em tokens exige uma curva de aprendizado inicialmente íngreme e uma mudança profunda na cultura organizacional de toda a equipe. Manter esse ecossistema exige forte governança e alinhamento constante, limitando a liberdade individual dos programadores. Para projetos de pequeno porte, montar essa infraestrutura de automação pode configurar um pesado excesso de engenharia, consumindo tempo precioso e recursos financeiros sem entregar um retorno de investimento que se justifique a curto prazo.

Aplicação Prática (PoC)

O objetivo desta Prova de Conceito (PoC) não foi o desenvolvimento de um sistema completo, mas sim a demonstração isolada e funcional da tecnologia de *Design Tokens*. A aplicação consiste em um painel interativo de partículas flutuantes sobreposto por um quadro translúcido, onde o usuário pode alternar a tematização global do sistema em tempo real.



Ambiente de Desenvolvimento

Para atestar a premissa de que a tecnologia atua ativamente na camada de estilo, optou-se por não utilizar *frameworks* robustos (como React ou Angular) ou pré-processadores. O ambiente foi estruturado da maneira mais fundamental possível:

- **Linguagens:** HTML5 (estruturação), CSS3 puro (dicionário de *tokens* e animações) e JavaScript (acionamento de eventos).
- **Editor e Execução:** Visual Studio Code para a escrita do código e navegadores web padrão (Google Chrome/Microsoft Edge) para a execução e renderização da interface.

Demonstração de Código

A tecnologia de Design Tokens foi implementada através de variáveis nativas do CSS. O código foi estruturado centralizando todas as definições visuais na pseudo-classe `:root`, que atua como o dicionário semântico primário do modo claro (*Light Mode*).

```
CSS
/* Dicionário Centralizado (Light Mode) */

:root {

  --bg-page: #f0f9ff;

  --text-main: #0f172a;

  --btn-primary: #0ea5e9;

  --glass-bg: rgba(255, 255, 255, 0.85);

  /* Cores das Partículas */

  --p-color-1: #ef4444;

}
```

A alteração da identidade visual completa do sistema (para o *Dark Mode*) é feita de forma escalável sobrescrevendo o dicionário quando o atributo `data-theme="dark"` é injetado na raiz do documento.

CSS

```
/* Sobrescrita de Tokens (Dark Mode) */

[data-theme="dark"] {

  --bg-page: #020617;

  --text-main: #f8fafc;

  --btn-primary: #8b5cf6;

  --glass-bg: rgba(15, 23, 42, 0.85);

  /* Partículas alteradas para Neon */

  --p-color-1: #fca5a5;

}
```

Dessa forma, o JavaScript (motor lógico) atua apenas como um "interruptor" para injetar o atributo, provando que a lógica de programação do software fica totalmente isolada das mudanças drásticas de requisitos visuais.

Relato de Experiência

Durante a concepção desta PoC, o objetivo central era provar a eficiência dos Design Tokens na tematização de interfaces. Para o conceito visual, idealizei a aplicação de uma paleta de cores de alto contraste (tons pastéis no modo claro e cores Neon no modo escuro), aplicada a um painel com efeito translúcido e partículas flutuantes no plano de fundo.

Para agilizar a etapa de codificação, utilizei ferramentas de Inteligência Artificial como apoio na configuração inicial do ambiente, solicitando a geração da estrutura básica e a digitação do dicionário base de variáveis visuais. A IA demonstrou ser uma excelente ferramenta de produtividade para tarefas repetitivas.

Entretanto, a ferramenta falhou ao tentar propor a lógica de interação dinâmica para a animação das partículas. A IA sugeriu e codificou uma abordagem onde o algoritmo central calculava e injetava os valores de cor diretamente nas propriedades individuais de cada componente visual em tempo real. Ao testar essa solução gerada pela máquina, me deparei com uma falha severa de arquitetura e desempenho. O sistema

apresentou travamentos constantes devido à sobrecarga de processamento na atualização de tela. Além disso, a abordagem da IA violava o princípio clássico da Separação de Responsabilidades, pois misturava regras puramente visuais dentro do motor lógico da aplicação.

Diante desse defeito arquitetural, decidi descartar a lógica proposta pela IA e reestruturar o projeto por conta própria. Mudei a abordagem para garantir que toda a animação e o mapeamento de cores das partículas fossem governados exclusivamente pela camada de estilo do sistema (arquivos de formatação visual). Dessa forma, reduzi o código lógico à sua função ideal que era atuar unicamente como um "interruptor" de estado que altera a classe base do documento.

Esse episódio ilustra de forma prática os desafios da Engenharia de Software moderna. A experiência validou que, embora a Inteligência Artificial seja útil para gerar a sintaxe inicial, a tomada de decisão arquitetural para garantir a confiabilidade, a performance e a correta aplicação dos padrões tecnológicos continua sendo uma responsabilidade inegociável do Engenheiro de Software.

Perspectivas Futuras e Mercado

Tendências

A evolução no uso de Design Tokens aponta para uma padronização universal e para a automação extrema. Atualmente, o *W3C Design Tokens Community Group* trabalha na criação de uma especificação padrão independente de plataforma, o que permitirá que qualquer ferramenta de design ou de engenharia leia e exporte essas variáveis nativamente.

Outra tendência arquitetural forte é a integração dos Design Tokens diretamente nas esteiras de integração e entrega contínuas. Em um futuro próximo, o fluxo de atualização visual será totalmente automatizado, onde uma alteração de cor feita por um designer em uma ferramenta gráfica (como o Figma) disparará automaticamente um evento (como um *Pull Request* no GitHub), atualizando e compilando o código do sistema em produção sem a necessidade de intervenção manual da equipe de desenvolvimento.

Adoção na Indústria

A adoção de Design Tokens deixou de ser uma tendência restrita a nichos e tornou-se um padrão consolidado na indústria de base tecnológica. Organizações que mantêm ecossistemas complexos de produtos, como Google (Material Design), IBM (Carbon Design System), Uber e Spotify, utilizam a tecnologia como o alicerce de suas interfaces.

No mercado de trabalho, há uma demanda crescente e acelerada por Engenheiros de *Frontend* e *UX Engineers* que dominem a construção e a manutenção de Design Systems escaláveis. As empresas buscam esses profissionais pois a aplicação correta dos tokens reduz drasticamente o débito técnico, o custo de manutenção corretiva e o tempo de lançamento de novas funcionalidades.

Conclusão

A prospecção tecnológica e a Prova de Conceito (PoC) desenvolvidas neste trabalho atestam que os Design Tokens representam uma solução altamente viável, robusta e madura para os desafios contemporâneos da Engenharia de Software. Ao atuarem como uma fonte única da verdade, os tokens previnem as falhas históricas de comunicação e retrabalho entre as equipes de concepção visual e os engenheiros de desenvolvimento.

Como demonstrado na PoC, o uso dessa tecnologia força a aplicação do princípio arquitetural de Separação de Responsabilidades. Ao isolar estritamente as regras de negócio e de interação (motor lógico) do dicionário semântico de estilização (camada visual), obtém-se um software altamente coeso e de baixo acoplamento. Conclui-se que o investimento na estruturação inicial de Design Tokens compensa o esforço, pois resulta em sistemas com facilidade de manutenção, excelente performance computacional e total aderência a mudanças contínuas de requisitos no ciclo de vida do software.

Referências

- SOMMERVILLE, Ian. *Engenharia de software*. 9. ed. São Paulo: Pearson, 2011.
- IEEE COMPUTER SOCIETY. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide): Version 3.0*. Piscataway: IEEE Computer Society, 2014.
- W3C DESIGN TOKENS COMMUNITY GROUP. *Design Tokens Format Module*. Disponível em: <https://www.designtokens.org/tr/drafts/format/>. Acesso em: 20 maio. 2026.
- FROST, Brad. *Atomic Design*. Massachusetts: Brad Frost, 2016.
- SUAREZ, Marco et al. *Design Systems Handbook*. New York: InVision, 2017.
- NORMAN, Donald A. *O design do dia a dia*. Rio de Janeiro: Rocco, 2006.